EventHandler

System Description

Abstract

This document describes the EventHandler support system of the Eclipse Arrowhead 5.2.2 framework, as implemented in `ah5-EventHandler-java-spring` of the primary Java source tree (`Eclipse-Arrowhead-java`) and deployed with the standalone Compose definition of that folder.

Contents

1 Overview **3**
1.1 Significant Prior Art . . . 4
1.2 How This System Is Meant to Be Used . . . 4
1.3 System functionalities and properties . . . 4
1.4 Important Delimitations . . . 5

2 Services **6**
2.1 Produced services . . . 6
2.2 Consumed services . . . 6

3 Security **7**
3.1 Security model . . . 7

4 References **7**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the EventHandler system. EventHandler provides publish-subscribe messaging in the local cloud: publishers publish typed events, subscribers receive them at their HTTPS endpoints, filtered by metadata and validity window.



Figure 1: SysML-style block view of EventHandler with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

Successor of the Arrowhead 4.6 EventHandler.

1.2 How This System Is Meant to Be Used

Subscribers `subscribe`; publishers `publish`; the EventHandler posts each matching event with its certificate and identity token. Operators manage subscriptions with `eventHandlerManagement`.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Event publication with metadata and timestamp tolerance.
- Subscriptions with metadata filter and validity window.
- HTTPS callback delivery; expired-subscription cleanup.

1.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8455	HTTPS port.
`authentication.policy`	declared	Standalone deployment: `outsourced` with Authentication identity tokens and mutual TLS.
`management.policy` / `management.whitelist`	authorization	Access to the management services.
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist check of requesters.
`service.authorization.enabled`	false	Per-call ConsumerAuthorization check (`authorization.verify`, cached 30 s, fail closed); standalone deployment: true.
`event.publish.tolerance.seconds`	60	Accepted clock skew of event timestamps.
`event.delivery.attempts`	1	Delivery attempts per notification.
`event.expiration.cleanup.interval`	60000	Cleanup interval (ms).
`normalization.mode`	extended	Name normalization.
`sysmonitor.enabled`, `sysmonitor.broker.*`, `sysmonitor.ssl.*`	true, amqp-broker:5671, server.ssl stores	Native `sysMonitor` publisher of the common library.
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (standalone deployment: `/crl/ca.crl`).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; the standalone deployment reads them from per-deployment Compose file secrets under `/run/secrets`.

The standalone deployment of this folder uses `config/application.properties` (copied into the image) with `server.port`=8455, `authentication.policy`=`outsourced`, `service.interface`.

`policy=CERT_AUTH`, `management.policy=sysop-only`, `service.authorization.enabled=true`, `mqtt.api.enabled=false`, `arrowhead.ssl.crl-file=/crl/ca.crl`; secrets are read from Compose file secrets under `/run/secrets` (`spring.config.import=configtree:/run/secrets/`). `enable.blacklist.filter` is not enabled there, so incoming requests are not checked against the Blacklist; blacklisted providers are still removed by DynamicServiceOrchestration.

1.3.3 Data stored by the system

Event subscriptions and log entries (database `ah_eventhandler`); events are not stored.

1.3.4 Non functional properties

Delivery is asynchronous.

1.3.5 Stateful or stateless

Stateful: subscriptions are persistent.

1.4 Important Delimitations

No event history or replay; delivery is best effort.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`eventPublish`	1.0.0	HTTP(S)	[1]	[2]
`eventSubscription`	1.0.0	HTTP(S)	[3]	[4]
`eventHandlerManagement`	1.0.0	HTTP(S)	[5]	[6]
`generalManagement`	1.0.0	HTTP(S)	[7]	[8]
`sysMonitor`	1.0.0	AMQP(S)	[9]	[10]

The HTTP interfaces are always offered and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	systemDiscovery, serviceDiscovery	register, revoke, lookup	Registration of the system and its services; lookup of the bootstrap services.	always
Authentication	identity	identity-login, identity-verify	Own identity token; verification of requester tokens.	`authentication.policy=outsourced`
Consumer Authorization	authorization	verify	Per-call authorization of its services.	`service.authorization.enabled=true`
Subscribers	notify endpoint	HTTPS POST	Event delivery.	subscriptions
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services, which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for 300 s); every such call needs a ConsumerAuthorization grant.

3 Security

The system can run in an insecure mode (`server.ssl.enabled=false`, plain HTTP, identity declared by the requester) for development, and in the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The standalone deployment of this folder (`config/application.properties` and `compose.yaml`) runs the secure mode.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates; MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the standalone deployment. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate`, `outsourced` (identity token verified with Authentication) or `declared` (insecure); the standalone deployment uses `outsourced`. An identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION`); with `service.authorization.enabled=true` ConsumerAuthorization decides each call; an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- With TLS the `notifyUri` must be HTTPS, so the EventHandler's token never travels in clear text.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "eventPublish – Service Description (EventHandler)," Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-eventPublish-SD.pdf.

[2] ——, "eventPublish HTTP/TLS/JSON – Interface Design Description (EventHandler)," Eclipse Arrowhead, IDD 5.2.2, 2026, eventHandler-v5.2.2-eventPublish-HTTP-IDD.pdf.

[3] ——, "eventSubscription – Service Description (EventHandler)," Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-eventSubscription-SD.pdf.

[4] ——, "eventSubscription HTTP/TLS/JSON – Interface Design Description (EventHandler)," Eclipse Arrowhead, IDD 5.2.2, 2026, eventHandler-v5.2.2-eventSubscription-HTTP-IDD.pdf.

[5] ——, "eventHandlerManagement – Service Description (EventHandler)," Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-eventHandlerManagement-SD.pdf.

[6] ——, "eventHandlerManagement HTTP/TLS/JSON – Interface Design Description (EventHandler)," Eclipse Arrowhead, IDD 5.2.2, 2026, eventHandler-v5.2.2-eventHandlerManagement-HTTP-IDD.pdf.

[7] ——, "generalManagement – Service Description (EventHandler)," Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-generalManagement-SD.pdf.

[8] ——, "generalManagement HTTP/TLS/JSON – Interface Design Description (EventHandler)," Eclipse Arrowhead, IDD 5.2.2, 2026, eventHandler-v5.2.2-generalManagement-HTTP-IDD.pdf.

[9] ——, "sysMonitor – Service Description (EventHandler)," Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-sysMonitor-SD.pdf.

[10] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (EventHandler),” Eclipse Arrowhead, IDD 5.2.2, 2026, eventHandler-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	